

Primera Suite

Mathematical Reference

v0.6.0

This document describes what the code does, not what it ought to do. Where a formulation is a deliberate approximation or carries a known artefact, that is stated rather than smoothed over.

Contents

1	Shared Primitives	3
1.1	Luminance	3
1.2	HSV Conversion	3
1.3	Soft Gamut Squeeze	3
1.4	Smoothstep	3
1.5	Skintone Masks	4
2	Transfer Functions	5
2.1	ARRI LogC3	5
2.2	ARRI LogC4	5
2.3	RED Log3G10	5
2.4	Sony S-Log3	5
2.5	ACEScct	5
2.6	DaVinci Intermediate	5
2.7	Kodak Cineon	5
2.8	Fuji F-Log2	5
2.9	Panasonic V-Log	6
3	Primera — Primary Grading	7
3.1	Exposure	7
3.2	Black Point	7
3.3	Temperature and Tint	7
3.4	Contrast	7
3.5	Shadow Fill	7
3.6	Highlight Gain	8
3.7	Roll Off (Soft Highlight)	8
3.8	Negative Saturation	8
3.9	Positive Saturation	8
3.10	Luminance Preservation	8
4	PrimeraHue — Tetrahedral Hue/Density	9
4.1	Corner Computation (Rodrigues Rotation)	9
4.2	Tetrahedral Interpolation	9
4.3	Cinecolor	9
4.4	Skintone Protection	10
5	PrimeraSkin — Skintone Sculpting	11
5.1	Evenness (Hue Compression)	11
5.2	Spatial Mask Pooling (Soften)	11
5.3	HSV Adjustments	12

5.4 Separation (Inverse-Mask Colour Contrast)	12
5.5 Mask-Scaled Gamut Squeeze	13
5.6 Show Mask Overlay and Legend	13
6 PrimeraSplit — Subtractive Split-Toning	15
6.1 Shadow/Highlight Weighting	15
6.2 Subtractive Colour Model	15
6.3 Luminance Compensation	15
6.4 Overlays	16

1 Shared Primitives

The following operations are shared across multiple tools via code fragments. Each tool is assembled by concatenating the fragments it needs, so a tool only carries the primitives it actually uses.

1.1 Luminance

Two luminance functions are used throughout. Rec. 709 (BT.709) is the default; Rec. 2020 (BT.2020) is used in PrimeraSplit where DaVinci Wide Gamut is assumed.

$$Y_{709} = 0.2126r + 0.7152g + 0.0722b$$

$$Y_{2020} = 0.2627r + 0.6780g + 0.0593b$$

1.2 HSV Conversion

Standard hexagonal-model conversion between RGB and HSV. Given $C_{\max} = \max(r, g, b)$, $C_{\min} = \min(r, g, b)$, and $\Delta = C_{\max} - C_{\min}$:

$$H = \begin{cases} 0 & \text{if } \Delta = 0 \\ \frac{1}{6} \cdot \frac{g-b}{\Delta} & \text{if } C_{\max} = r \\ \frac{1}{6} \cdot \left(\frac{b-r}{\Delta} + 2 \right) & \text{if } C_{\max} = g \\ \frac{1}{6} \cdot \left(\frac{r-g}{\Delta} + 4 \right) & \text{if } C_{\max} = b \end{cases}$$

with H wrapped to $[0, 1)$, then:

$$S = \begin{cases} \Delta/C_{\max} & \text{if } C_{\max} > 0 \\ 0 & \text{otherwise} \end{cases}, \quad V = C_{\max}$$

The inverse follows the standard piecewise chroma/secondary construction.

No clamping is applied in either direction. For a pixel with $C_{\min} < 0 < C_{\max}$ — possible in a wide-gamut working space — this yields $S > 1$, and the inverse reconstructs the original triple exactly, since the achromatic offset $m = V - VS = C_{\min}$ carries the negative part through. Hue rotations performed in this extended space therefore stay bounded within $[C_{\min}, C_{\max}]$ rather than diverging. A pixel with $C_{\max} \leq 0$ has no recoverable hue and is treated as achromatic.

1.3 Soft Gamut Squeeze

Applied per-channel. A piecewise function that compresses the shoulder via $\tanh$ and the toe via exponential roll-off, keeping the mid-range on the identity. With knee $k = 0.9$ and range $r = 0.1$ for the shoulder, and toe threshold $t_0 = 0.1$:

$$f(x) = \begin{cases} k + r \cdot \tanh\left(\frac{x-k}{r}\right) & \text{if } x > k \\ t_0 \cdot e^{x/t_0-1} & \text{if } x < t_0 \\ x & \text{otherwise} \end{cases}$$

The shoulder asymptotically approaches $k + r = 1.0$; the toe asymptotically approaches 0. PrimeraSkin applies a mask-scaled variant (Section 5.5).

1.4 Smoothstep

The Hermite basis used for all smooth interpolation (shadow/highlight blends, skintone masks, split-toning crossfade):

$$\text{smoothstep}(a, b, x) = t^2(3 - 2t), \quad t = \text{clamp}\left(\frac{x-a}{b-a}, 0, 1\right)$$

1.5 Skintone Masks

Two distinct masks exist, and they do not share a detection strategy. The HSV-domain mask (Section 1.5.1) is used by PrimeraHue for Cinecolor holdout; the chromaticity-domain mask (Section 1.5.2) is used throughout PrimeraSkin. Both are anchored at the nominal skin hue

$$H_0 = 28^\circ/360^\circ \approx 0.0778, \quad W_0 = 28^\circ/360^\circ, \quad w = W_0 \cdot \rho$$

where ρ is the user's Range parameter (default 1.0), and both use the wrapped hue distance

$$d(h) = \min(|h - H_0|, 1 - |h - H_0|).$$

1.5.1 HSV Mask

A hue gate multiplied by an HSV saturation gate:

$$M_{\text{hue}(h)} = 1 - \text{smoothstep}(0, w, d(h))$$

$$M_{\text{sat}(s)} = \text{smoothstep}(0.08, 0.35, s)$$

$$M_{\text{hsv}(r,g,b)} = M_{\text{hue}(H)} \cdot M_{\text{sat}(S)}$$

with $M_{\text{hsv}} = 0$ where $S < 10^{-6}$ or $V < 10^{-6}$. PrimeraHue calls this with $\rho = 1$.

1.5.2 Chromaticity Mask, Value-Gated

PrimeraSkin detects skin in **rg-chromaticity** rather than by HSV saturation. Normalising by the channel sum removes overall brightness, so the surviving ratios describe the reflectance rather than the exposure — which is what makes a single gate work across fair and dark skin instead of tracking lightness. In log-space, skin clusters tightly around $g_n \approx 0.33$ with red leading blue. With $\Sigma = r + g + b$ (and the mask defined as 0 when $\Sigma < 0.01$):

$$r_n = \frac{r}{\Sigma}, \quad g_n = \frac{g}{\Sigma}, \quad b_n = \frac{b}{\Sigma}$$

$$M_{g_n} = \text{smoothstep}(0.22, 0.33, g_n) \cdot (1 - \text{smoothstep}(0.37, 0.48, g_n))$$

$$M_{rb} = \text{smoothstep}(0, 0.04, r_n - b_n)$$

$$M_{\text{chroma}} = M_{g_n} \cdot M_{rb}$$

The ramp widths above are deliberately wide. Narrow ramps sharpen the mask edge, which sounds desirable but makes the Density slider blotchy: the gate then resolves grain as structure.

Two value gates give the user Low Gate (l) and High Gate (h) control, for holding off crushed shadows and specular highlights:

$$M_{\text{lo}(v)} = \begin{cases} \text{smoothstep}(l/2, l, v) & \text{if } l > 0.001 \\ 1 & \text{otherwise} \end{cases}$$

$$M_{\text{hi}(v)} = \begin{cases} 1 - \text{smoothstep}(h - \frac{1-h}{2}, h, v) & \text{if } h < 0.999 \\ 1 & \text{otherwise} \end{cases}$$

The hue gate is retained on top of the chromaticity gate so that Range remains a meaningful user control:

$$M_{\text{gated}(r,g,b)} = M_{\text{hue}(H)} \cdot M_{\text{chroma}} \cdot M_{\text{lo}(V)} \cdot M_{\text{hi}(V)}$$

where H and V come from the HSV conversion of (r, g, b) .

2 Transfer Functions

All tools that operate in log-space use a common set of encode/decode functions mapping between scene-linear and the selected log encoding. Mid-grey (0.18 linear) is the reference exposure throughout.

Each function below is the *encode* direction (linear $\rightarrow$ log). The decode is the algebraic inverse.

2.1 ARRI LogC3

$$f(x) = \begin{cases} c \cdot \log_{10}(ax + b) + d & \text{if } x > \text{cut} \\ ex + f & \text{otherwise} \end{cases}$$

with $a = 5.555556$, $b = 0.052272$, $c = 0.247190$, $d = 0.385537$, $e = 5.367655$, $f = 0.092809$, $\text{cut} = 0.010591$.

2.2 ARRI LogC4

Let $a = \frac{262144-16}{117.45}$, $b = \frac{1023-95}{1023}$, $c = \frac{95}{1023}$.

$$s = \frac{7 \ln 2 \cdot 2^{7-14c/b}}{ab}, \quad t = \frac{2^{14(-c/b)+6} - 64}{a}$$

$$f(x) = \begin{cases} \frac{\log_2(ax + 64) - 6}{14} \cdot b + c & \text{if } x \geq t \\ \frac{x-t}{s} & \text{otherwise} \end{cases}$$

2.3 RED Log3G10

$$f(x) = \begin{cases} 0.224282 \cdot \log_{10}(155.975327(x + 0.01) + 1) & \text{if } x + 0.01 \geq 0 \\ 15.1927(x + 0.01) & \text{otherwise} \end{cases}$$

2.4 Sony S-Log3

$$f(x) = \begin{cases} \frac{420 + 261.5 \log_{10}((x + 0.01)/0.19)}{1023} & \text{if } x \geq 0.01125 \\ \frac{x(171.2102946929 - 95)/0.01125 + 95}{1023} & \text{otherwise} \end{cases}$$

2.5 ACEScct

$$f(x) = \begin{cases} \frac{\log_2(x)+9.72}{17.52} & \text{if } x > 2^{-7} = 0.0078125 \\ 10.5402377x + 0.0729055 & \text{otherwise} \end{cases}$$

2.6 DaVinci Intermediate

$$f(x) = \begin{cases} C(\log_2(x + A) + B) & \text{if } x > \text{cut}_{\text{lin}} \\ Mx & \text{otherwise} \end{cases}$$

with $A = 0.0075$, $B = 7$, $C = 0.07329248$, $M = 10.44426855$, $\text{cut}_{\text{lin}} = 0.00262409$.

2.7 Kodak Cineon

$$f(x) = \frac{300 \log_{10}(x(1 - \delta) + \delta) + 685}{1023}, \quad \delta = 0.0108$$

2.8 Fuji F-Log2

$$f(x) = \begin{cases} c \cdot \log_{10}(ax + b) + d & \text{if } x \geq \text{cut} \\ ex + f & \text{otherwise} \end{cases}$$

with $a = 5.555556$, $b = 0.064829$, $c = 0.245281$, $d = 0.384316$, $e = 8.799461$, $f = 0.092864$,
cut = 0.000889.

2.9 Panasonic V-Log

$$f(x) = \begin{cases} c \cdot \log_{10}(x + b) + d & \text{if } x \geq 0.01 \\ 5.6x + 0.125 & \text{otherwise} \end{cases}$$

with $b = 0.00873$, $c = 0.241514$, $d = 0.598206$.

3 Primera — Primary Grading

The primary tool applies a chain of per-channel operations in a fixed order. Let (r, g, b) denote the working pixel value, initially in the selected log encoding. The chain is: decode $\rightarrow$ exposure $\rightarrow$ black point $\rightarrow$ temp/tint $\rightarrow$ encode $\rightarrow$ contrast $\rightarrow$ shadows $\rightarrow$ highlights $\rightarrow$ roll-off $\rightarrow$ negative saturation $\rightarrow$ positive saturation. Exposure through temp/tint are therefore linear-light operations; everything after the re-encode is a log-domain operation.

3.1 Exposure

Scene-linear gain in photographic stops:

$$(r, g, b)_{\text{lin}} = 2^n \cdot \text{decode}(r, g, b)$$

where $n \in [-6, 6]$ is the exposure parameter in stops.

3.2 Black Point

Smooth compression of the toe region of each channel (in linear), using an exponential roll-off below a knee:

$$\text{bp}(x) = \begin{cases} x - \beta & \text{if } x - \beta \geq \kappa \\ \kappa \cdot e^{(x-\beta)/\kappa-1} & \text{if } x > 0 \\ x & \text{otherwise} \end{cases}$$

where $\beta = -p_{\text{bp}}$ (the user's Black Point slider, ± 0.05) and $\kappa = 0.005$.

3.3 Temperature and Tint

Per-channel linear-light gains. Temperature shifts along the blue–amber axis; tint along green–magenta:

$$r' = r \cdot 2^\tau, \quad g' = g \cdot 2^\varphi, \quad b' = b \cdot 2^{-\tau}$$

where τ is the temperature parameter and φ is the tint parameter. The pixel is then re-encoded to the working log space.

3.4 Contrast

A pivoted power function that preserves 0, 1, and the pivot point p . The pivot defaults to $\text{encode}(0.18)$ (the transfer function's mid-grey) plus a user offset, clamped to $[0.01, 0.99]$. For input $x \in [0, 1]$:

$$\text{contrast}(x) = \begin{cases} p \cdot \left(\frac{x}{p}\right)^\gamma & \text{if } x \leq p \\ 1 - (1 - p) \cdot \left(\frac{1 - x}{1 - p}\right)^\gamma & \text{if } x > p \end{cases}$$

where $\gamma \in [0.5, 2]$ is the contrast parameter. This is symmetric about the pivot: shadows and highlights are stretched or compressed equally. Values outside $[0, 1]$ pass through unchanged.

3.5 Shadow Fill

Below mid-grey $m = \text{encode}(0.18)$, a gain is blended from full effect at black to no effect at mid-grey, using the smoothstep basis as the crossfade:

$$t = \frac{x}{m}, \quad s(t) = t^2(3 - 2t)$$

$$\text{shadow}(x) = x \cdot (G + (1 - G) \cdot s(t)), \quad 0 < x < m$$

where $G = 2^{p_{\text{shadows}}}$. Pixels at or above mid-grey pass through unchanged.

3.6 Highlight Gain

The mirror of Shadow Fill, operating between mid-grey and 1.0:

$$t = \frac{x - m}{1 - m}, \quad s(t) = t^2(3 - 2t)$$

$$\text{highlight}(x) = x \cdot (1 + (G - 1) \cdot s(t)), \quad m < x < 1$$

where $G = 2^{p_{\text{highlights}}}$.

3.7 Roll Off (Soft Highlight)

A tanh-based shoulder compression above a variable knee point. The knee descends from 1.0 toward mid-grey as the amount increases; beyond amount = 1.0, a strength multiplier steepens the compression:

$$\kappa = 1 - \min(\alpha, 1) \cdot (1 - m), \quad \sigma = 1 + \max(\alpha - 1, 0) \cdot 2$$

$$\text{rolloff}(x) = \begin{cases} \kappa + (1 - \kappa) \cdot \frac{\tanh\left(\frac{\sigma(x - \kappa)}{1 - \kappa}\right)}{\sigma} & \text{if } x > \kappa \\ x & \text{otherwise} \end{cases}$$

where $\alpha \in [0, 2]$ is the Roll Off amount and $m = \text{encode}(0.18)$.

3.8 Negative Saturation

Multiplicative desaturation toward Rec. 709 luminance, in the log-encoded domain. With saturation factor $s \in [0, 1]$ (default 1, i.e. no change):

$$c' = Y + s(c - Y), \quad c \in \{r, g, b\}$$

where $Y = Y_{709(r, g, b)}$. At $s = 0$ the image is monochrome. The operation is skipped entirely if any channel is negative, since the luminance reference is not meaningful there.

3.9 Positive Saturation

Operates in HSV space. The S channel is boosted by a power-of-two scale and clipped:

$$S' = \min(S \cdot 2^{p_{\text{sat}}}, 1)$$

then converted back to RGB. As above, skipped if any channel is negative.

3.10 Luminance Preservation

An optional post-step (checkbox) applied after either saturation operation. Scales all channels uniformly to restore pre-adjustment luminance:

$$(r', g', b') = \frac{Y_{\text{before}}}{Y_{\text{after}}} \cdot (r, g, b)$$

applied only where both luminances are positive.

4 PrimeraHue — Tetrahedral Hue/Density

Performs per-channel hue rotation and density adjustment via tetrahedral interpolation. The order of operations is: corner computation → optional hard squeeze of the corners → interpolation → luminance preservation → Cinecolor → optional soft squeeze.

Tetrahedral interpolation is only defined on the unit cube, so **any pixel with a channel outside $[0, 1]$ is returned unmodified**. The tool expects a log-encoded signal, not scene-linear.

4.1 Corner Computation (Rodrigues Rotation)

Each of the six cube corners (R, G, B, C, M, Y) is rotated around the achromatic axis by a hue angle $\theta = h \cdot 60^\circ$ (where $h \in [-1, 1]$ is the slider value, so ± 1 covers $\pm 60^\circ$). For a corner with identity color $\mathbf{v} = (v_r, v_g, v_b)$:

$$\mu = \frac{v_r + v_g + v_b}{3}, \quad \mathbf{c} = \mathbf{v} - \mu \mathbf{1}$$

The centred vector is rotated around the $(1, 1, 1)$ axis. Because $\mathbf{c}$ is already perpendicular to that axis, the general Rodrigues formula collapses to its first two terms, with the cross product reducing to differences of components:

$$c'_r = c_r \cos \theta + k(c_b - c_g) \sin \theta$$

$$c'_g = c_g \cos \theta + k(c_r - c_b) \sin \theta$$

$$c'_b = c_b \cos \theta + k(c_g - c_r) \sin \theta$$

with $k = 1/\sqrt{3} \approx 0.57735$. The density offset d is then added uniformly:

$$\mathbf{v}' = \mathbf{c}' + (\mu + d) \mathbf{1}$$

Density makes the shifted color subjectively more “colorful” without adding energy — it uniformly offsets all three channels at the given corner.

The **hard squeeze** option clamps each rotated corner to $[0, 1]$ at this point, before interpolation. It acts on the corner definitions, not on the pixel.

4.2 Tetrahedral Interpolation

The RGB unit cube is decomposed into six tetrahedra by sorting the channel values. Each tetrahedron shares the black corner $(0, 0, 0)$ and white corner $(1, 1, 1)$ and includes two of the six chromatic corners. For a pixel with $r \geq g \geq b$ (the R–Y tetrahedron):

$$\mathbf{o} = r\mathbf{R}_c + g(\mathbf{Y}_c - \mathbf{R}_c) + b(\mathbf{1} - \mathbf{Y}_c)$$

The five other sort orderings produce analogous expressions using the corresponding pair of adjacent corners. This is the same interpolation structure used in 3D LUT evaluation and described by Yedlin’s DisplayPrep approach.

4.3 Cinecolor

A fixed tetrahedral remap that emulates the two-color subtractive process of Cinecolor (c. late 1930s–early 1950s). With blend parameter $t \in [0, 1]$, the chromatic corners are set to:

$$\mathbf{R}_c = (1, 0, 0), \quad \mathbf{G}_c = (0, 1, t), \quad \mathbf{B}_c = (0, 0, 1 - t)$$

$$\mathbf{C}_c = (0, 1, 1), \quad \mathbf{M}_c = (1, 0, 1 - t), \quad \mathbf{Y}_c = (1, 1, t)$$

At $t = 0$ the mapping is identity; at $t = 1$ the blue channel is fully redistributed toward green and away from magenta, emulating orthochromatic film’s insensitivity to red light and the resulting blue–orange palette of the duplitized print process. The pixel is hard-clamped to $[0, 1]$ for this stage regardless of the hard squeeze setting.

4.4 Skintone Protection

When Cinecolor is active, an optional holdout matte centered on the skin hue protects the blue channel — and only the blue channel, since that is the only one Cinecolor redistributes:

$$b_{\text{out}} = b_{\text{cinecolor}}(1 - M_{\text{hsv}}) + b_{\text{in}} \cdot M_{\text{hsv}}$$

where M_{hsv} is the HSV mask of Section 1.5.1, evaluated on the clamped pixel.

5 PrimeraSkin — Skintone Sculpting

Dedicated skintone adjustments operating in HSV, gated by the chromaticity mask of Section 1.5.2. Because both HSV and rg-chromaticity are purely geometric decompositions of RGB, PrimeraSkin is encoding-agnostic in form — but the gate thresholds were tuned against log-encoded material, so it expects a log timeline in practice.

Unlike the other three tools, PrimeraSkin uses the **texture** variant of the DCTL entry point, giving it access to neighbouring pixels. That access exists for one reason: the mask is otherwise a pure function of a single pixel’s RGB, which is the root of the graininess discussed in Section 5.2.

The order of operations is:

1. Evenness hue compression (unpooled, broadened mask)
2. Own mask $m_{\text{own}} = M_{\text{gated}}$ on the possibly-Evenness-adjusted pixel
3. Spatial pooling $\rightarrow$ effective mask m (Section 5.2)
4. Separation gate mask m_{sep} (Section 5.4)
5. Hue / Saturation / Density adjustments, scaled by m
6. Legend strip, then Show Mask overlay (either returns early)
7. Separation hue field
8. Mask-scaled soft squeeze

5.1 Evenness (Hue Compression)

An optional hue compression step pulls skin-adjacent hues toward the median skin hue H_0 , emulating the evening-out of skintones done by hair and makeup on set. It uses a broadened mask (range $\rho + 0.4$) to catch outlier hues at the edges:

$$d = H - H_0 \quad (\text{wrapped to } (-0.5, 0.5])$$

$$d' = d(1 - \alpha \cdot M_{\text{gated}}^*), \quad H' = H_0 + d'$$

where $\alpha \in [0, 1]$ is the Evenness parameter and M_{gated}^* is the mask of Section 1.5.2 evaluated with the broadened range. This step is deliberately **not** pooled: it is hue compression rather than a level adjustment, so it does not exhibit the artefact that motivates pooling, and pooling it would double the tap cost for no benefit.

5.2 Spatial Mask Pooling (Softening)

The per-pixel mask is a pure function of one pixel’s RGB. Sensor grain therefore dithers borderline skin pixels back and forth across the gate edges, leaving a stochastic salt of under-adjusted pixels inside otherwise solid skin — visible as a pointillist texture when Density is pushed. The artefact is spatial, so the fix must be spatial.

Let $R = p_{\text{soften}} \cdot W \cdot 0.004$ be the pooling radius in pixels (W = frame width, giving $\approx 15\text{px}$ at UHD and full strength — resolution-independent by construction). The mask is sampled over a sparse pattern of the centre plus three concentric rings of $N_i = (8, 12, 16)$ taps at radii $\rho_i = (\frac{1}{3}, \frac{2}{3}, 1) \cdot R$. Each ring’s starting phase is offset by the golden angle $\Phi \approx 2.39996$ rad so that taps never align on image axes, which would produce directional banding:

$$(x_{ij}, y_{ij}) = \left(X + \rho_i \cos\left(i\Phi + \frac{2\pi j}{N_i}\right), Y + \rho_i \sin\left(i\Phi + \frac{2\pi j}{N_i}\right) \right)$$

Each tap carries a spatial Gaussian weight and a loose bilateral term in rg-chromaticity, so that pooling does not drag in a background of a different colour. With $\sigma_s = R/2$, $\sigma_c = 0.06$, and chromaticity coordinates $g_n = g/\Sigma$, $\delta_n = (r - b)/\Sigma$:

$$w_{\text{spatial}(i)} = e^{-\rho_i^2/2\sigma_s^2}$$

$$w_{\text{sim}} = \exp\left(-\left((g_n^{\text{tap}} - g_n^{\text{ctr}})^2 + (\delta_n^{\text{tap}} - \delta_n^{\text{ctr}})^2\right)/2\sigma_c^2\right)$$

The centre tap enters with weight 1 and the pooled estimate is the weighted mean

$$m_{\text{pool}} = \frac{m_{\text{own}} + \sum_{i,j} w_{ij} M_{\text{gated}}(\mathbf{p}_{ij})}{1 + \sum_{i,j} w_{ij}}$$

which is then combined with the pixel’s own mask by a **soft union** rather than a replacement:

$$m = 1 - (1 - m_{\text{own}})(1 - m_{\text{pool}})$$

This is the load-bearing choice. Because $m \geq \max(m_{\text{own}}, m_{\text{pool}})$, pooling can only **raise** the mask: it fills the grain holes it was introduced to fix and can never erode a confident edge. At $p_{\text{soften}} = 0$ no taps are made at all and the tool is bit-identical to its per-pixel form.

5.3 HSV Adjustments

All three adjustments are applied in HSV, scaled by the effective mask m :

- **Hue rotation:** $H' = H - m \cdot \Delta H / 360$ ($\Delta H \in [-20^\circ, 20^\circ]$)
- **Saturation:** $S' = \max(S(1 + m\sigma), 0)$ ($\sigma \in [-0.5, 0.5]$)
- **Density:** $V' = \max(V(1 - m\delta), 0)$ ($\delta \in [-0.2, 0.2]$; positive = darker)

The sign of the hue rotation matches the legend and overlay convention: negative slider values move toward green, positive toward magenta.

5.4 Separation (Inverse-Mask Colour Contrast)

Separation acts on everything the rest of the tool ignores. It defines a smooth vector field on the hue circle with a stable attractor at the complement of the skin vector and an unstable equilibrium at the skin hue itself:

$$h_c = H_0 + \frac{1}{2} \quad (\approx 208^\circ, \text{the cyan/teal band})$$

$$\Delta h = -\omega \cdot K \cdot \sin(2\pi(h - h_c)), \quad K = 0.075$$

The sinusoidal form is chosen over a naive “rotate toward the target by the shortest path” for two reasons. First, skin sits at the field’s zero, so skin-hued pixels receive no push **before** any mask is applied. Second, naive rotation tears the hue circle at the target’s antipode — which is precisely the skin hue — sending adjacent pixels in opposite directions around the circle; the sinusoidal field is smooth there instead. The field reaches its maximum $K = 0.075$ (a 27° shift, capping migration at roughly 30%) at the 90° flanks, 118° and 298° .

The gate weight is

$$\omega = p_{\text{sep}} \cdot (1 - m_{\text{sep}}) \cdot \text{smoothstep}(0.08, 0.35, S) \cdot \text{smoothstep}(0.01, 0.05, V)$$

Each factor answers a specific failure of the naive inverse mask:

- ($1 - m_{\text{sep}}$) m_{sep} must be a **pooled** mask. Inverting the raw per-pixel mask would not merely skip grain-dithered borderline skin pixels — it would actively push them toward teal, producing cyan stippling inside faces. Since the soft union of Section 5.2 can only raise the mask, it can only lower its inverse, so pooling can never let the push leak inward. Accordingly the gate pools at $\max(p_{\text{soften}}, 0.5)$, independently of whether Soften itself is in use, and never writes back to m .

Chroma weight $1 - m$ includes neutrals, which have no business migrating to teal. Weighting by saturation makes the operation **redistribute** colour that is already present rather than inventing it.

Value ramp M_{gated} returns 0 for $\Sigma < 0.01$, which inverts to a full push on near-blacks. Since $\Sigma < 0.01$ implies $V \leq 0.01$, this ramp closes that hole exactly.

The field is evaluated on the pixel’s **native** hue — its value before the Hue and Evenness adjustments — so that the repeller stays anchored to real skin even when those controls have rotated masked pixels away from H_0 .

Because the operation is a pure hue rotation at constant S and V , it cannot leave the range spanned by the original channels and needs no containment of its own.

Known artefact. Reds sit on the magenta flank of the repeller and drift the long way round: crimson at 355° takes -14.7° on an unmasked saturated pixel at full strength, landing near 340° . This is the classic teal-and-orange casualty and it arrives at roughly half the available dose. No protection notch is applied, since one would also flatten the field for the warm practicals and skin-adjacent wood and beige that the effect exists to move.

5.5 Mask-Scaled Gamut Squeeze

The final squeeze is scaled by the mask rather than applied flat:

$$c_{\text{out}} = c + \min(4m, 1) \cdot (f(c) - c), \quad c \in \{r, g, b\}$$

with f the soft squeeze of Section 1.3. The reasoning: the squeeze exists to contain adjustment overshoot, and adjustment magnitude already scales with m , so a pixel only faintly pooled into the mask should not receive the full shoulder and toe. Without this, Soften paints a visible squeeze halo around subjects against bright or dark backgrounds. The gain of 4 reaches full squeeze by $m = 0.25$, leaving core-skin behaviour unchanged.

5.6 Show Mask Overlay and Legend

Both the overlay and the legend are renderings of a single zone function, so the strip is a closed-form key to what the overlay does. The signed hue distance from skin centre is normalised by the mask width:

$$z = \text{clamp}\left(d_{\text{signed}} \frac{H}{w}, -1, 1\right)$$

Zone colour is a per-channel gain applied to a base colour, continuous at $z = 0$:

$$g(z) = \begin{cases} (1.4, 1.3 - 0.8|z|, 0.5 + 0.8|z|) & \text{if } z \leq 0 \\ (1.4 - 0.9z, 1.3, 0.5 + 0.8z) & \text{if } z > 0 \end{cases}$$

$$\text{zone}(z, \mathbf{v})_c = v_c \cdot g_c(z), \quad c \in \{r, g, b\}$$

so $z < 0$ (hue clockwise of skin centre) reads magenta, $z = 0$ gold, and $z > 0$ (counter-clockwise) reads green/cyan.

For the **overlay**, the base is the pixel’s own value $\mathbf{v} = (V, V, V)$, so the false colour rides on image luminance, and the blend uses the squared mask:

$$\mathbf{c}_{\text{out}} = V\mathbf{1} + m^2(\text{zone}(z, V\mathbf{1}) - V\mathbf{1})$$

Squaring compresses marginal matches toward grey while keeping confident skin vivid. Zone classification uses the **original** pixel hue so that the overlay stays stable as the Hue slider

moves, while the mask strength is taken from the adjusted pixel so the overlay still responds to the sliders.

For the **legend**, the base is a fixed representative skin value $(0.55, 0.42, 0.25)$ and z is swept linearly across the bottom 7.5% of frame, $z = 1 - 2x_n$, so the strip reads GREEN $\rightarrow$ SKIN $\rightarrow$ MAG. from left to right — matching the direction of the Hue slider.

Show Mask returns before Separation is applied: it is a readout of the skin mask, not a preview of the graded result.

6 PrimeraSplit — Subtractive Split-Toning

Applies zone-weighted subtractive colour adjustments to shadows and highlights independently, operating directly on the log-encoded signal.

6.1 Shadow/Highlight Weighting

The image luminance (Y_{2020} , Section 1.1) determines the blend between shadow and highlight zones via the smoothstep crossfade:

$$w_{\text{hi}(x)} = \text{smoothstep}\left(p - \frac{\tau}{2}, p + \frac{\tau}{2}, x\right), \quad w_{\text{shd}(x)} = 1 - w_{\text{hi}(x)}$$

where $p = \text{encode}(0.18) + \text{offset}$ is the effective pivot and τ is the transition softness. The two weights are complementary by construction: $w_{\text{shd}} + w_{\text{hi}} = 1$.

Each zone’s six slider values are scaled by that zone’s weight and summed, giving a single pair of adjustment vectors per pixel:

$$\mathbf{a}_{\text{rgb}} = w_{\text{shd}} \mathbf{a}_{\text{shd,rgb}} + w_{\text{hi}} \mathbf{a}_{\text{hi,rgb}}, \quad \mathbf{a}_{\text{cmy}} = w_{\text{shd}} \mathbf{a}_{\text{shd,cmy}} + w_{\text{hi}} \mathbf{a}_{\text{hi,cmy}}$$

6.2 Subtractive Colour Model

For each primary, a *positive* slider value subtracts the complementary channels (adding the colour by removing its complement), while a *negative* value subtracts the primary itself. This is what makes the model subtractive: colour is only ever added by taking something away.

Slider	Positive (> 0)	Negative (< 0)
Red a_r	$g' = g(1 - a_r), \quad b' = b(1 - a_r)$	$r' = r(1 + a_r)$
Green a_g	$r' = r(1 - a_g), \quad b' = b(1 - a_g)$	$g' = g(1 + a_g)$
Blue a_b	$r' = r(1 - a_b), \quad g' = g(1 - a_b)$	$b' = b(1 + a_b)$
Cyan a_c	$r' = r(1 - a_c)$	$g' = g(1 + a_c), \quad b' = b(1 + a_c)$
Magenta a_m	$g' = g(1 - a_m)$	$r' = r(1 + a_m), \quad b' = b(1 + a_m)$
Yellow a_y	$b' = b(1 - a_y)$	$r' = r(1 + a_y), \quad g' = g(1 + a_y)$

All six adjustments are applied multiplicatively in sequence, so they compose rather than average.

6.3 Luminance Compensation

Subtractive adjustments inherently darken the image. An estimated “subtractive impact” is accumulated from the adjustment magnitudes, weighted by how much each direction actually removes — 0.5 where the adjustment subtracts from two channels, 0.33 where it subtracts from one:

$$I = 0.5 \sum_c (\max(a_{c,\text{rgb}}, 0) + \max(-a_{c,\text{cmy}}, 0)) + 0.33 \sum_c (\max(-a_{c,\text{rgb}}, 0) + \max(a_{c,\text{cmy}}, 0))$$

A compensation gain is derived from it and clamped, then applied in proportion to the Preserve Luminance slider:

$$\kappa = \text{clamp}(1 + 2I, 1, 2)$$

$$(r, g, b)_{\text{out}} = (r, g, b) \cdot (1 + (\kappa - 1)\lambda)$$

where $\lambda \in [0, 1]$ is the Preserve Luminance slider (default 0.8). Note this is an open-loop estimate from the slider values, not a measured luminance ratio as in Section 3 — it compensates the adjustment, not the pixel.

6.4 Overlays

Three optional overlays share the toning maths rather than approximating it:

Show Ramp a horizontal greyscale ramp x_n is pushed through the **identical** pipeline — weighting, subtractive adjustment, and luminance compensation — and drawn as a band at a user-positioned height. It therefore shows exactly what the tool does at every tonal position, not a model of it.

Show Curve the same processed ramp is plotted per channel as three curves, by marking pixels where the channel value crosses the row's normalised height.

Show Pivot draws the shadow and highlight weighting curves and a vertical marker at the effective pivot, visualising τ directly.

Show Chart draws the shared per-transfer-function step chart, graduated in stops.